
API Documentation

Chat completions, streaming, and the model catalogue

Protocol	OpenAI-compatible HTTP + SSE
Base URL	<code>https://ai.zeiko.biz.id/v1</code>
Auth	<code>Authorization: Bearer <API key></code>

Point any OpenAI-compatible client at the base URL and it works unchanged. This document is the complete contract: every field you can send, every field you get back, and every error you can be given.

Contents

- 1. Base URL and endpoints 3
- 2. Authentication 3
- 3. Chat completions 3
- 4. Identifying your end users 5
- 5. The response 5
- 6. Streaming 7
- 7. The model catalogue 8
- 8. Errors 11
- 9. Limits 12
- 10. Examples 13

1. Base URL and endpoints

```
base_url = https://ai.zeiko.biz.id/v1
```

Every path works with and without the `/v1` prefix. Some clients append `/v1` themselves, others are given a base URL that already ends in it. Both hit the same handler, so neither setup is wrong.

Method	Path	Key?	What it does
GET	<code>/health</code>	no	Liveness, and how many models are available.
GET	<code>/v1/models</code> <code>/models</code>	yes	The model catalogue. See section 7.
GET	<code>/v1/models/{id}</code> <code>/models/{id}</code>	yes	One model's entry, or <code>404</code> . An id containing a <code>/</code> works as written, unescaped.
POST	<code>/v1/chat/completions</code> <code>/chat/completions</code>	yes	Chat, streaming and non-streaming. The main endpoint.
POST	<code>/v1/completions</code> <code>/completions</code>	yes	Legacy text completions.
POST	<code>/v1/embeddings</code> <code>/embeddings</code>	yes	Embeddings, on models that offer them.
OPTIONS	any path above except <code>/health</code>	no	CORS preflight.

```
GET /health → {"status":"ok","models":3}
```

2. Authentication

Every endpoint except `/health` and CORS preflight takes your API key as a bearer token:

```
Authorization: Bearer Kunci-Zeiko-xxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx
```

Keys are the literal prefix `Kunci-Zeiko-` followed by a version-4 UUID, so a key is only ever hex digits and hyphens and can be pasted anywhere without being mangled. A key sent bare, with no `Bearer` scheme in front of it, is also accepted, because some clients send it that way.

Your key is issued to you directly. Keep it server-side: anyone holding it can spend against your account. If you believe it has been exposed, ask us for a replacement and the old one stops working immediately.

The model catalogue needs the key too. `GET /v1/models` is authenticated. A client that fetches the model list before it has been given credentials gets `401`, not an empty list. That is the usual cause of a mysterious `401` on an otherwise working setup.

3. Chat completions

`POST /v1/chat/completions` takes the ordinary OpenAI request body. Two fields are required: `model` and `messages`.

```
{
  "model": "Wissangeni-512B-V1",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "stream": true,
  "temperature": 0.7,
  "max_tokens": 512,
  "user_id": "tenant-42:user-9f3c1ab7"
}
```

Parameter	Notes
<code>model</code>	Required. An id from <code>/v1/models</code> . Matched exactly and case-sensitively. A name we issued you that is not in the listing still resolves to the entry it was set up for, but the listed id is the one to integrate against.
<code>messages</code>	Required. The usual <code>role</code> / <code>content</code> array.
<code>stream</code>	<code>true</code> returns SSE, <code>false</code> a single JSON body. Every model supports both.
<code>user_id</code>	Who this request is for. <code>user</code> is read as the same field, so an OpenAI client's own <code>user=</code> argument works without <code>extra_body</code> . See section 4: read it.
<code>max_tokens</code> <code>max_completion_tokens</code>	Clamped down to the model's <code>max_completion_tokens</code> , never refused for being too large. Whichever name you send is the one kept. Omit them and the model's own ceiling applies.
<code>stop</code>	Up to four entries, as OpenAI.
<code>reasoning_effort</code>	<code>none</code> , <code>minimal</code> , <code>low</code> , <code>medium</code> , <code>high</code> , <code>max</code> , on models whose catalogue entry has a <code>reasoning</code> block. It can change the price; see <code>pricing</code> in section 7. Also read from <code>reasoning.effort</code> , <code>thinking.effort</code> , the boolean <code>enable_thinking</code> , and a budget in <code>thinking.budget_tokens</code> .
<code>reasoning.exclude</code> <code>include_reasoning</code>	Leave the reasoning trace out of the reply. Either can only remove a trace; neither turns on one a model does not expose.
<code>temperature</code> , <code>top_p</code> , <code>top_k</code> , <code>min_p</code> , <code>presence_penalty</code> , <code>frequency_penalty</code> , <code>repetition_penalty</code> , <code>seed</code> , <code>logit_bias</code> , <code>logprobs</code> , <code>top_logprobs</code>	Standard sampling and scoring parameters.
<code>response_format</code> , <code>tools</code> , <code>tool_choice</code>	JSON mode and tool calling, on models that list them. A model that can hold a reply to a schema also lists <code>structured_outputs</code> .
<code>stream_options</code>	Ignored. Usage always arrives on the final chunk, so <code>include_usage</code> is never needed.

Check `supported_parameters` before relying on a parameter. Each model's catalogue entry lists exactly what it accepts. Sending something not on that list is not an error; it simply has no effect.

Other endpoints

`/v1/completions` takes the legacy prompt-style body and is billed identically. `/v1/embeddings` takes `model` plus `input` (a string or an array of strings) and returns the standard embeddings response. Neither streams.

4. Identifying your end users

Send one field, `user_id`, on every request. It is a stable, opaque identifier for the person the request is for.

```
{
  "model": "Wissangeni-512B-V1",
  "messages": [ ... ],
  "user_id": "tenant-42:user-9f3c1ab7"
}
```

That is the whole integration. There is nothing else to set, no header to add, and nothing to configure on your side. We take that one value and handle everything downstream of it.

One alternative spelling is worth knowing: `user`, the field an OpenAI client already sends natively, is read as the same value, so `user="tenant-42:user-9f3c1ab7"` does the job without `extra_body`. It wins if both arrive. Pick one spelling and keep to it — sending a different one on a different call is a different partition, and every cache hit is lost.

Why it matters

`user_id` is the prompt-cache partition key. Two things follow from it, and both are worth money:

- **Isolation.** Requests carrying different `user_id` values can never reuse each other's cached prompt prefix. Without the field, everyone on your API key shares a single partition, so one customer's prefix becomes reusable by another customer's request.
- **Cost and speed.** A cached prefix is billed at the cache-read rate, a small fraction of the fresh-input rate, and it removes most of the time-to-first-token on a long system prompt. A `user_id` that changes between requests throws every cache hit away; a stable one keeps them.

You can see it working: `usage.prompt_tokens_details.cached_tokens` appears on the response once a prefix is being reused, and is absent while it is not.

Choosing the value

Rule	Why
Stable	The same person gets the same string on every request, across sessions and restarts. This is what makes the cache hit.
Unique	Two different people must never share one value, or they share a cache partition.
Opaque	A hash, a UUID, or an internal account id. Never an email address, phone number, username or real name.
Short and ASCII	Up to 120 characters. Keep well under it.

Omitting `user_id` is allowed and the request still succeeds. You simply get no isolation and, in practice, far fewer cache hits.

5. The response

```
{
  "id": "32dadfde-f3dd-4ad8-acc5-387b72415f46",
  "object": "chat.completion",
  "created": 1789936200,
  "model": "Wissangeni-512B-V1",
}
```

```

"choices": [
  {
    "index": 0,
    "message": {
      "role": "assistant",
      "content": "Hello! How can I help you today?",
      "reasoning_content": "The user greeted me...",
      "tool_calls": []
    },
    "finish_reason": "stop",
    "native_finish_reason": "stop"
  }
],
"usage": {
  "prompt_tokens": 18,
  "completion_tokens": 256,
  "total_tokens": 274,
  "prompt_tokens_details": { "cached_tokens": 12 },
  "completion_tokens_details": { "reasoning_tokens": 64 },
  "cost": 0.102949871
}
}

```

Field	Type	Meaning
<code>id</code>	string	A UUIDv4 for this request, repeated on every frame of a stream and returned as the <code>X-Request-Id</code> response header. Log it: quoting it lets us find the exact request.
<code>object</code>	string	<code>"chat.completion"</code> , or <code>"chat.completion.chunk"</code> on a stream frame.
<code>created</code>	integer	Unix seconds, stamped when the request arrived.
<code>model</code>	string	The model id you asked for.
<code>choices[].message.content</code>	string	The answer.
<code>choices[].message.reasoning_content</code>	string	The reasoning trace, on models that expose one. Absent when empty or excluded.
<code>choices[].message.tool_calls</code>	array	Assembled tool calls, each <code>{id, type, function:{name, arguments}}</code> . Absent when there are none.
<code>choices[].finish_reason</code>	string	<code>stop</code> , <code>length</code> , <code>tool_calls</code> .
<code>choices[].native_finish_reason</code>	string	The same value again, under the name some OpenRouter-shaped clients read. Never a second, different reason — read either one.

Usage and cost

`prompt_tokens` counts **your own messages only**, measured as they arrived. Anything added on your behalf is not charged to you, so the figure matches what you sent.

Field	Type	Meaning
<code>usage.prompt_tokens</code>	integer	Input tokens.
<code>usage.completion_tokens</code>	integer	Output tokens, reasoning included.
<code>usage.total_tokens</code>	integer	The two above, added.

Field	Type	Meaning
<code>usage.prompt_tokens_details.cached_tokens</code>	integer	Cache-hit portion of the input. Never larger than <code>prompt_tokens</code> . Absent when zero.
<code>usage.completion_tokens_details.reasoning_tokens</code>	integer	Reasoning portion of the output. Absent when zero.
<code>usage.cost</code>	number	What this request cost, in USD, to nine decimal places. Also repeated as <code>usage.usage</code> for clients that read that name. One number, never two prices. Both are absent together on a model carrying no published price at all.

Nine decimal places is deliberate: a short request at a tenth of a dollar per million tokens comes to a few millionths of a cent, and rounding to six would report it as free. Parse it as a decimal, not a float, if you are reconciling invoices.

Response headers

Header	Value
<code>X-Request-Id</code>	Same UUIDv4 as <code>id</code> , on streaming and non-streaming answers alike. <code>id</code> in the body is the copy that is always there, so read it from there if you record only one. An error carries neither — a request refused before it reached a model never got an id — so quote the time, the model and the status instead.
<code>Content-Type</code>	<code>application/json</code> , or <code>text/event-stream; charset=utf-8</code> when streaming.
<code>Retry-After</code>	Seconds to wait. Sent with <code>429</code> and <code>503</code> .

6. Streaming

Send `"stream": true` and the response is `text/event-stream`: one `data:` frame per chunk, then a final frame carrying usage, then `[DONE]`.

```
data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1",
      "choices":[{"index":0, "delta":{"role":"assistant"}, "finish_reason":null}]}

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1",
      "choices":[{"index":0, "delta":{"content":"Hel"}, "finish_reason":null}]}

: still working on it

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1",
      "choices":[{"index":0, "delta":{"content":"lo!"}, "finish_reason":"stop"}]}

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1", "choices":[],
      "usage":{"prompt_tokens":18, "completion_tokens":256, "total_tokens":274,
              "cost":0.010293}}

data: [DONE]
```

(Wrapped for print. On the wire each `data:` payload is a single line, and every frame ends with a blank line.)

What a client must handle

Frame	Rule
First frame	Carries <code>delta.role: "assistant"</code> and usually no content. Do not assume it has text.
Content frames	<code>delta.content</code> is a string fragment; concatenate in arrival order. A reasoning trace, where a model exposes one, arrives as <code>delta.reasoning_content</code> alongside it.
Tool calls	<code>delta.tool_calls</code> arrives as partial fragments keyed by <code>index</code> : accumulate <code>function.arguments</code> across frames before parsing.
Comment lines	A line beginning with <code>:</code> is an SSE comment sent to hold the connection open during a quiet moment. Discard it. Whatever follows the colon is free text and will change; never match on it, and never try to JSON-parse it. A compliant SSE parser already discards these; a hand-rolled one that only reads lines starting <code>data:</code> is also fine.
Final usage frame	Carries <code>usage</code> , and it always arrives. Detect it by the presence of <code>usage</code>, not by an empty <code>choices</code> : it usually has <code>choices: []</code> , but the last frame can instead carry both <code>usage</code> and a final choice at once. So a client has to do two things — read <code>usage</code> off whichever frame carries it, and guard <code>choices[0]</code> rather than assume it exists.
Error frame	If a request fails part-way through, a frame carrying <code>{"error": {"message": "...", "type": "server_error"}}</code> is sent before <code>[DONE]</code> , so a stream that has started never just ends in silence. The HTTP status stays <code>200</code> , because it was sent before anything went wrong.
<code>data: [DONE]</code>	The terminator. Always last, and not JSON.

Practical notes

- `id`, `created` and `model` are identical on every frame of one response.
- Each choice carries `native_finish_reason` beside `finish_reason`, always the same value. Read either; do not expect them to differ.
- Frames can split across TCP reads. Buffer until a blank line before parsing, and accept both `\n\n` and `\r\n\r\n` as the separator.
- A multi-byte character is never split across two frames' `delta.content`.
- Use `curl -N`, and turn off response buffering in your HTTP client.
- Set your client timeout above the maximum request duration in section 9. With keep-alives, an idle-read timeout will not fire spuriously, but a total-request timeout can.
- Disconnecting mid-stream stops the work. You are charged for what was generated up to that point.

7. The model catalogue

`GET /v1/models` and `GET /models` return the identical document. `/v1/models/{id}` returns a single entry.

```
curl https://ai.zeiko.biz.id/v1/models \
-H "Authorization: Bearer Kunci-Zeiko-xxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx"
```

Before you call it

1. **It needs your API key.** Unauthenticated it answers `401`.
2. **These ids are what `model` should be set to.** Matching is exact and case-sensitive. A name issued to you that is not in the listing still resolves, but the listed id is the one to integrate against.

3. **Re-fetch periodically.** The catalogue reflects what is currently offered and can change; do not cache it for the lifetime of a process.
4. **Prices are decimal strings in USD per single token**, not JSON numbers, and not per million. `"0.000000003"` loses its last digits through a float.
5. **A missing `pricing` object means no published price**, not a free model.
6. **These models are sold at three prices, not one.** Which one you pay is decided by the `reasoning_effort` you send, before anything else is read. `pricing.bands` publishes all three; the top-level rates are the `default` band, and `pricing.default_band` names the one a request that sends no `reasoning_effort` at all lands on.
7. **A field that is absent was never set.** An unset `description`, `context_length`, `max_completion_tokens` or `default_parameters` is left out rather than published as `""`, `0` or `null`.

```
{
  "object": "list",
  "data": [
    {
      "id": "Wissangeni-512B-V1",
      "object": "model",
      "name": "Wissangeni 512B V1",
      "display_name": "Wissangeni 512B V1",
      "owned_by": "ZeikoAI",
      "created": 1789900000,
      "description": "Long-context assistant model.",
      "context_length": 128000,
      "max_completion_tokens": 8192,
      "architecture": {
        "modality": "text->text",
        "input_modalities": ["text"],
        "output_modalities": ["text"]
      },
      "pricing": {
        "prompt": "0.00000015",
        "completion": "0.0000006",
        "input_cache_read": "0.00000003",
        "internal_reasoning": "0.0000006",
        "default_band": "default",
        "bands": {
          "non_thinking": {
            "name": "Non-thinking", "efforts": ["none","minimal"],
            "prompt": "0.00000015", "completion": "0.0000005",
            "input_cache_read": "0.00000003", "internal_reasoning": "0.0000005"
          },
          "default": {
            "name": "Default", "efforts": ["low","medium","high"],
            "prompt": "0.00000015", "completion": "0.0000006",
            "input_cache_read": "0.00000003", "internal_reasoning": "0.0000006"
          },
          "max": {
            "name": "Max", "efforts": ["max"],
            "prompt": "0.00000015", "completion": "0.0000009",
            "input_cache_read": "0.00000003", "internal_reasoning": "0.0000009"
          }
        }
      },
      "overrides": [
        { "utc_days": ["monday","friday"], "utc_start": 100, "utc_end": 400,
          "prompt": "0.0000003", "completion": "0.0000012",
          "input_cache_read": "0.00000006" }
      ]
    },
    {
      "supported_parameters": ["frequency_penalty","max_tokens","reasoning",
        "reasoning_effort","response_format","seed","stop",
        "temperature","tool_choice","tools","top_p"],
    }
  ]
}
```

```

    "default_parameters": { "temperature": 0.7 },
    "reasoning": {
      "mandatory": false,
      "default_enabled": true,
      "supported_efforts": ["max", "high", "medium", "low", "none"],
      "default_effort": "high"
    }
  }
}

```

Field	Meaning
<code>id</code>	The value to put in <code>model</code> .
<code>name</code> / <code>display_name</code>	Human-readable label.
<code>owned_by</code>	Who publishes the model.
<code>created</code>	Unix seconds.
<code>context_length</code>	Input ceiling in tokens. A prompt over it is refused with <code>413</code> , never truncated.
<code>max_completion_tokens</code>	Output ceiling. A larger <code>max_tokens</code> is clamped to this, not refused.
<code>architecture</code>	What the model takes in and gives back. <code>modality</code> is the two arrays joined, e.g. <code>"text->text"</code> .
<code>pricing.prompt</code>	Fresh input, USD per token, as a decimal string.
<code>pricing.completion</code>	Output, USD per token.
<code>pricing.input_cache_read</code>	Cache-hit input. Absent means a cache hit is not discounted, not that it is free.
<code>pricing.input_cache_write</code>	Cache write, where one is charged.
<code>pricing.internal_reasoning</code>	Reasoning tokens. Absent means they bill at the output rate.
<code>pricing.request</code>	A flat per-request amount, where there is one. Flat, not per token.
<code>pricing.bands</code>	Price by thinking effort. Three entries, cheapest first: <code>non_thinking</code> , <code>default</code> , <code>max</code> . Each carries a human <code>name</code> , the <code>efforts</code> that land a request on it, and a complete price — not only the rates that band moves. Published for all three even where two are the same number.
<code>pricing.default_band</code>	Which of the three a request that names no <code>reasoning_effort</code> is billed at. Usually <code>default</code> , but not necessarily — read it rather than assume it.
<code>pricing.overrides[]</code>	Time-of-day pricing. Optional <code>utc_days</code> (lowercase weekday names) and optional <code>utc_start</code> / <code>utc_end</code> as <code>HHMM</code> integers in UTC: <code>100</code> is 01:00, <code>1430</code> is 14:30. Half-open: start inclusive, end exclusive; an end before its start wraps midnight; no times means all day. Each override publishes a complete price , not only the fields it moves.
<code>supported_parameters</code>	Exactly what this model accepts, sorted.
<code>default_parameters</code>	Applied when you do not send them. Override any of them freely.
<code>reasoning</code>	Present only on models that reason . <code>supported_efforts</code> is the menu <code>reasoning_effort</code> accepts; <code>default_effort</code> is what you get by naming none. Its absence is how you detect a non-reasoning model.

Checking a bill against the catalogue

`usage.cost` is the published rates multiplied by the token counts on the same response. Cached input is carved out of `prompt_tokens` and reasoning out of `completion_tokens`, so nothing is counted twice:

```
cost = (prompt_tokens - cached_tokens) * pricing.prompt
      + cached_tokens * pricing.input_cache_read
      + (completion_tokens - reasoning_tokens) * pricing.completion
      + reasoning_tokens * pricing.internal_reasoning
      + pricing.request
```

Start from the band you asked for — `pricing.bands[...]` for the `reasoning_effort` you sent, or `pricing.bands[pricing.default_band]` if you sent none. Then use whichever `overrides` entry covers the UTC moment the request ran, and the standing rates otherwise.

One answer is not priced by its tokens, and it is the one that would otherwise look wrong. A reply that declines the request — the model saying it will not answer — is charged as a single flat amount instead of the tokens it took to say so. It is an ordinary `200` with ordinary token counts, and `usage.cost` carries that flat amount, so the formula above will not reproduce it. Reconcile from `usage.cost`, which is always the amount charged; the formula is how you check an answered request, not a substitute for the figure on the response.

8. Errors

```
{
  "error": {
    "message": "model \"gpt-9\" is not available",
    "type": "invalid_request_error",
    "code": "model_not_found"
  }
}
```

Status	code	When
400	none	Malformed JSON, or a missing or empty <code>model</code> .
400	<code>invalid_request</code>	A body this model will not accept. Alone among the <code>4xx</code> answers this one's <code>type</code> reads <code>server_error</code> , so branch on the status and the <code>code</code> , never on <code>type</code> .
401	<code>invalid_api_key</code>	Missing or unknown key. Also what an unauthenticated <code>GET /v1/models</code> gets.
403	<code>invalid_api_key</code>	The key is disabled.
403	<code>model_forbidden</code>	Your key is not entitled to that model.
403	none	Type <code>forbidden</code> , message <code>blocked</code> : this address is not served. Nothing about the request will change it.
404	<code>model_not_found</code>	Unknown model id.
404	<code>not_found</code>	No such path.
413	<code>context_length_exceeded</code>	The prompt is over the model's <code>context_length</code> . The message gives both figures.
413	<code>too_large</code>	The request is too large for the model to take.

Status	code	When
413	no body at all	The request body is over the size limit in section 9. This one is refused before it can be read, so it is the single case that comes back as <code>text/plain</code> rather than the JSON shape above. Guard your error parsing for it — a client that calls <code>JSON.parse</code> on every error body throws here.
429	<code>rate_limit_exceeded</code>	A per-minute rate limit or a daily quota is spent. The per-minute refusal carries <code>Retry-After</code> .
500	<code>internal_error</code>	Something went wrong on our side before an answer existed. Rare, always worth reporting.
502	<code>model_unavailable</code>	The model could not answer. Retry with backoff; if it persists, contact us with the model id and roughly when it started — an error carries no request id of its own.
503	<code>overloaded</code>	Temporarily at capacity. Carries <code>Retry-After</code> .
504	<code>timeout</code>	The model took too long to answer.

Retry policy

- 429 and 503: honour `Retry-After` when present, otherwise exponential backoff from about one second.
- 502 and 504: retry two or three times with backoff.
- 400, 401, 403, 404, 413: do not retry. Fix the request or the credentials.
- 500: retry once, then report it. A second one on the same request is not a blip.

9. Limits

Limit	Value	Behaviour at the edge
Request body size	20 MB	413 before the body is read, as <code>text/plain</code> rather than the JSON error shape — see section 8. Comfortably above a full-context request: at typical text density, even a million tokens of JSON is a few megabytes.
Input tokens	per model, as <code>context_length</code>	413. Refused, never truncated.
Output tokens	per model, as <code>max_completion_tokens</code>	A larger <code>max_tokens</code> is clamped down, not refused.
Requests per minute	per key	429 with <code>Retry-After</code> .
Requests per day	per key	429. Resets daily.
Tokens per day	per key	429. Input and output counted together.
Maximum request duration	10 minutes	504. Set your client timeout above this.

There is no message-count limit of its own: a conversation is bounded by the model's `context_length` and by the body size above. Your per-key rate limits and quotas are part of your agreement with us. Ask if you need them raised.

10. Examples

Streaming, curl

```
curl -N https://ai.zeiko.biz.id/v1/chat/completions \
-H "Authorization: Bearer Kunci-Zeiko-xxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx" \
-H "Content-Type: application/json" \
-d '{
  "model": "Wissangeni-512B-V1",
  "messages": [{"role": "user", "content": "Hello!"}],
  "user_id": "tenant-42:user-9f3c1ab7",
  "stream": true
}'
```

Non-streaming, OpenAI Python SDK

```
from openai import OpenAI

client = OpenAI(
    base_url="https://ai.zeiko.biz.id/v1",
    api_key="Kunci-Zeiko-xxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx",
)

reply = client.chat.completions.create(
    model="Wissangeni-512B-V1",
    messages=[{"role": "user", "content": "Hello!"}],
    extra_body={"user_id": "tenant-42:user-9f3c1ab7"},
)

print(reply.id)                # == the X-Request-Id header
print(reply.choices[0].message.content)
print(reply.usage.prompt_tokens, reply.usage.completion_tokens)
print(reply.usage.model_extra.get("cost"))
```

Streaming, OpenAI Python SDK

```
stream = client.chat.completions.create(
    model="Wissangeni-512B-V1",
    messages=[{"role": "user", "content": "Write a haiku."}],
    extra_body={"user_id": "tenant-42:user-9f3c1ab7"},
    stream=True,
)

usage = None
for chunk in stream:
    if chunk.usage is not None:      # take usage off whichever frame carries it
        usage = chunk.usage         # and keep going: it may carry text too
    if not chunk.choices:           # guard - never assume choices[0] exists
        continue
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="", flush=True)

print("\n", usage)
```

Streaming, Node with fetch

```
const res = await fetch("https://ai.zeiko.biz.id/v1/chat/completions", {
  method: "POST",
  headers: {
    "Authorization": "Bearer Kunci-Zeiko-xxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx",
```

```

    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    model: "Wissangeni-512B-V1",
    messages: [{ role: "user", content: "Hello!" }],
    user_id: "tenant-42:user-9f3c1ab7",
    stream: true,
  }),
});

const reader = res.body.getReader();
const decoder = new TextDecoder();
let buf = "";

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buf += decoder.decode(value, { stream: true });

  // Frames are separated by a blank line; tolerate CRLF from proxies.
  let m;
  while ((m = /\r?\n\r?\n/.exec(buf)) !== null) {
    const frame = buf.slice(0, m.index);
    buf = buf.slice(m.index + m[0].length);

    for (const line of frame.split(/\r?\n/)) {
      if (line.startsWith(":")) continue; // keep-alive comment
      if (!line.startsWith("data:")) continue;
      const data = line.slice(5).trim();
      if (data === "[DONE]") continue; // terminator, not JSON

      const json = JSON.parse(data);
      if (json.error) { console.error(json.error.message); continue; }
      if (json.usage) { console.log("\ncost:", json.usage.cost); }
      const piece = json.choices?.[0]?.delta?.content;
      if (piece) process.stdout.write(piece);
    }
  }
}

```

Listing models

```

curl https://ai.zeiko.biz.id/v1/models \
-H "Authorization: Bearer Kunci-Zeiko-xxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx"

```

Support. Every answer carries an `id` in its body, repeated as the `X-Request-Id` header. Log it. A question that quotes one can be answered precisely; one without usually cannot. An error never has one, so for those log the time, the model and the status instead.